

CHAPTER 1: INTRODUCTION

1.1 Background

Natural disasters such as floods, earthquakes, cyclones, landslides, fires, and severe storms can cause significant loss of life, infrastructure, and property. During such emergencies, the effectiveness of disaster management depends heavily on how quickly incidents are reported, verified, prioritized, and responded to. However, disaster response operations may involve multiple stakeholders such as citizens, disaster management officers, volunteers, hospitals, shelters, and administrators. When information is handled through disconnected systems, phone calls, messages, spreadsheets, or manual coordination, important information can be delayed, duplicated, or difficult to track.

A major challenge in disaster response is obtaining a clear and timely understanding of the situation. A citizen may report an incident with a location, description, and supporting evidence, but emergency authorities still need to verify the report and determine its severity and priority. They also need to identify appropriate resources such as rescue personnel, vehicles, equipment, hospitals, and shelters. Managing these activities manually becomes increasingly difficult when multiple incidents occur simultaneously.

The proposed **AI-Rescue: Intelligent Disaster Response & Resource Coordination Platform** is designed to provide a centralized web-based system for managing these activities. The platform allows citizens to report disaster incidents and enables authorized disaster management officers to review and verify the submitted information. Once an incident is verified, the system can use AI-assisted decision-support capabilities to assess its severity, recommend a priority level, and suggest the types of resources that may be required.

The platform also supports resource coordination, emergency communication, incident status tracking, dashboards, and report generation. Volunteers or responders can receive relevant assignments and notifications, while hospitals and shelters can provide availability information required for emergency coordination.

The system follows a human-in-the-loop approach. AI recommendations are intended to assist authorized officers rather than replace human decision-making. This allows the platform to combine automated analysis with human verification and operational judgment.

The proposed solution is intended as an academic project with an industry-oriented architecture. Its modular design provides a foundation for future integration with technologies such as live geographic information, IoT-based monitoring, weather services, mobile applications, external emergency-service systems, and advanced predictive analytics.

1.2 Objectives

The main objectives of the AI-Rescue platform are:

1. **To provide centralized disaster incident reporting** through which citizens can submit disaster-related information, location details, and supporting evidence.
 2. **To improve incident verification and management** by enabling authorized disaster management officers to review submitted incidents and update their status.
 3. **To provide AI-assisted decision support** for assessing incident severity, recommending priority levels, and suggesting required resource types.
 4. **To improve emergency resource coordination** by managing available resources, vehicles, equipment, responders, hospitals, and shelters.
 5. **To facilitate emergency communication** by providing relevant notifications and updates to citizens, responders, hospitals, shelters, and other authorized users.
 6. **To support real-time operational monitoring** through dashboards containing incident, resource, and response information.
 7. **To maintain records and accountability** through reporting and audit-related information.
 8. **To provide a scalable foundation** for future integration with IoT devices, mapping services, mobile applications, external emergency systems, and advanced AI capabilities.
-

1.3 Purpose, Scope, and Applicability

This section defines what the proposed AI-Rescue system is intended to achieve, what functions are included in the project, and where the system can be applied.

1.3.1 Purpose

The purpose of the AI-Rescue platform is to improve the organization and coordination of disaster response activities through a centralized digital system.

The platform aims to reduce delays caused by fragmented information and manual coordination by bringing incident reporting, verification, AI-assisted prioritization, resource coordination, emergency communication, response tracking, and reporting into a single system.

The system is designed to assist disaster management authorities in making informed decisions while maintaining human control over important operational decisions.

1.3.2 Scope

The scope of the proposed system includes:

- User registration and authentication.
- Role-based access for different categories of users.
- Disaster incident reporting by citizens.
- Collection of incident location, description, and supporting evidence.
- Incident validation and verification by authorized officers.
- Incident status management.
- AI-assisted severity assessment.
- AI-assisted priority recommendation.
- AI-assisted resource recommendation.
- Resource availability management.
- Vehicle and equipment management.
- Resource assignment to incidents.
- Volunteer/responder coordination.
- Hospital and shelter availability management.
- Emergency notifications.
- Incident response tracking.
- Operational dashboards.
- Report generation.
- Audit and system activity recording.

The initial project focuses on developing the **core software platform and decision-support functionality**. Advanced features such as physical IoT deployment, drone integration, large-scale government integration, and fully autonomous emergency decision-making are outside the initial implementation scope.

1.3.3 Applicability

The AI-Rescue platform can be applied in environments where multiple stakeholders need to coordinate disaster-related incidents and emergency resources.

Potential application areas include:

- District-level disaster management operations.
- Municipal or local emergency management.
- Flood response.

- Cyclone and storm response.
- Earthquake-related emergency coordination.
- Fire and rescue coordination.
- Landslide response.
- Community-level disaster reporting.
- Emergency shelter coordination.
- Hospital emergency-capacity coordination.

The system can be adapted according to the requirements of a particular geographical region or disaster-management organization.

For the academic implementation, the platform serves as a **prototype demonstrating how digital systems and AI-assisted decision support can improve disaster-response coordination** rather than claiming to replace existing government emergency infrastructure.

1.4 Achievements

Since this is a **proposed academic project**, we should be careful with this section.

We should **not falsely claim that the system has already reduced response time or successfully handled real disasters**.

Instead, document the achievements of the project/design work:

- Designed a centralized disaster-response management platform.
- Defined multiple user roles and role-based access requirements.
- Designed the complete disaster incident lifecycle from reporting to closure.
- Designed AI-assisted severity, priority, and resource recommendation functionality.
- Designed a structured relational database for managing project information.
- Defined system modules for incident management, AI analysis, resource coordination, emergency services, notifications, reporting, and auditing.
- Developed conceptual and system-design models including system flowcharts, ER diagrams, DFDs, use-case diagrams, and sequence diagrams.
- Defined security, data-integrity, and audit requirements.
- Established a modular architecture that can support future extensions.

CHAPTER 2: SURVEY OF TECHNOLOGIES

This chapter explains the technologies considered for developing the **AI-Rescue: Intelligent Disaster Response & Resource Coordination Platform**. The selected technologies are based on the functional requirements, scalability, security, database requirements, AI integration, and feasibility of implementation.

The project uses a combination of web development, backend programming, relational database management, and AI/ML technologies. Each technology has a specific role in the proposed system.

2.1 Frontend Technologies

2.1.1 HTML5

HTML5 is used to create the basic structure of the web application. It provides elements for forms, tables, navigation menus, dashboards, buttons, input fields, and other interface components.

In the AI-Rescue platform, HTML5 can be used for:

- User registration and login forms.
- Disaster incident reporting forms.
- Resource management interfaces.
- Hospital and shelter information pages.
- Officer dashboards.
- Report and notification interfaces.

HTML5 provides the structural foundation of the application's user interface.

2.1.2 CSS3

CSS3 is used to control the visual appearance and layout of the web application.

It can be used for:

- Responsive page layouts.
- Dashboard design.
- Forms and buttons.
- Navigation menus.
- Tables and cards.
- Status indicators.

- Mobile-friendly interfaces.

CSS allows the system to maintain a consistent and user-friendly interface across different screen sizes.

2.1.3 JavaScript

JavaScript provides client-side interactivity and dynamic behaviour.

In the AI-Rescue platform, JavaScript can be used for:

- Client-side Form validation.
- Interactive dashboard components.
- Dynamic incident information.
- Notification interfaces.
- Asynchronous requests to backend services.
- Interactive maps and location-related features.
- Updating information without unnecessarily reloading the entire page.

JavaScript therefore complements HTML and CSS by providing interactive functionality.

2.2 Backend Technologies

2.2.1 Java

Java is considered as the primary backend programming language for the system because of its object-oriented architecture, security features, portability, extensive libraries, and suitability for enterprise-level web applications.

Java can handle:

- Business logic.
- User authentication.
- Role-based access control.
- Incident processing.
- Resource management.
- Database communication.
- API development.
- Notification processing.

Java also provides a structured environment for developing a modular backend.

2.2.2 Spring Boot

Spring Boot can be used as the backend application framework for the proposed platform.

It simplifies the development of Java-based web applications and REST APIs by providing configuration and infrastructure required for application development.

For AI-Rescue, Spring Boot can manage:

- REST APIs.
- Authentication and authorization.
- Incident management services.
- Resource management services.
- Notification services.
- Database connectivity.
- Communication with the AI/ML service.

The use of Spring Boot also makes it easier to separate the application into independent modules.

2.3 Database Technology

2.3.1 PostgreSQL

PostgreSQL is selected as the relational database management system for the AI-Rescue platform.

The system handles structured and interconnected information such as users, roles, incidents, disaster types, resources, vehicles, equipment, assignments, hospitals, shelters, notifications, reports, audit records, and system settings. A relational database is therefore appropriate for maintaining consistency and relationships between these entities.

PostgreSQL provides features such as:

- Primary keys.
- Foreign keys.
- Unique constraints.
- Check constraints.
- Transactions.
- Referential integrity.
- Structured SQL queries.
- Support for complex relational data.

The database design of the project consists of the finalized **16-table V1 schema**.

The major tables include:

1. ROLE
2. USER
3. DISASTER_TYPE
4. INCIDENT
5. INCIDENT_MEDIA
6. AI_RECOMMENDATION
7. RESOURCE
8. VEHICLE
9. EQUIPMENT
10. RESOURCE_ASSIGNMENT
11. HOSPITAL
12. SHELTER
13. NOTIFICATION
14. REPORT_LOG
15. AUDIT_LOG
16. SYSTEM_SETTINGS

The database will serve as the primary persistent storage layer of the application.

2.4 AI and Machine Learning Technologies

2.4.1 Python

Python is suitable for implementing the AI/ML component because of its extensive machine-learning ecosystem and relatively simple syntax.

Python can be used as a separate AI service responsible for processing verified incident information.

Possible inputs include:

- Disaster type.
- Incident description.
- Number of people affected.
- Location-related information.
- Reported severity indicators.
- Other structured incident attributes.

The AI component can then provide decision-support outputs such as:

- Severity assessment.
 - Priority recommendation.
 - Required resource recommendation.
 - Confidence score.
-

2.4.2 Machine Learning Libraries

Python-based machine-learning libraries such as **scikit-learn** can be considered for implementing the initial AI component.

For the academic V1 system, the AI functionality should remain practical rather than pretending to have a sophisticated real-world disaster prediction model.

The AI module can initially demonstrate classification or scoring-based decision support using a suitable dataset or controlled project dataset.

The output is then sent back to the Java backend for officer review.

AI architecture

Verified Incident



Java Backend



Python AI Service



AI Analysis



Severity / Priority /

Resource Recommendation



Java Backend



Officer Review



Operational Decision

This is a much more defensible architecture in your viva than saying "AI automatically decides what emergency teams should do."

2.5 API Technology

2.5.1 REST API

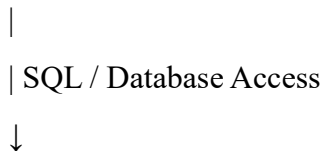
REST (Representational State Transfer) APIs are used as the communication mechanism between the major components of the proposed system.

The basic communication structure can be represented as:

Web Frontend



Java / Spring Boot Backend



PostgreSQL Database

The AI component communicates separately with the backend:

Java / Spring Boot Backend



Python AI Service



AI Recommendation



Java Backend

This approach keeps the AI component separate from the main application logic and allows the AI service to be improved independently.

2.6 System Modelling and Design Tools

2.6.1 Visual Paradigm

Visual Paradigm is used as a system modelling and diagram-design tool.

It can be used to create:

- Entity Relationship Diagrams.
- Use Case Diagrams.
- Sequence Diagrams.
- Data Flow Diagrams.
- System design models.
- UML diagrams.

Using standard diagram components and notation is important for presenting the system design clearly in the project documentation.

2.6.2 PostgreSQL Database Modelling Tools

Database modelling tools can be used to represent the PostgreSQL database structure and relationships between tables.

The database design will be based on the finalized 16-table schema, including primary keys, foreign keys, attributes, and integrity constraints.

The ER diagram will represent the logical relationships between the entities, while the actual PostgreSQL database will be used during implementation.

2.7 Version Control Technology

2.7.1 Git

Git is a distributed version-control system used to track changes made to the project source code and documentation.

It allows developers to:

- Maintain version history.
 - Track modifications.
 - Restore previous versions.
 - Work on different development stages.
 - Manage project files systematically.
-

2.7.2 GitHub

GitHub can be used as the remote repository for storing and managing the project's source code and documentation.

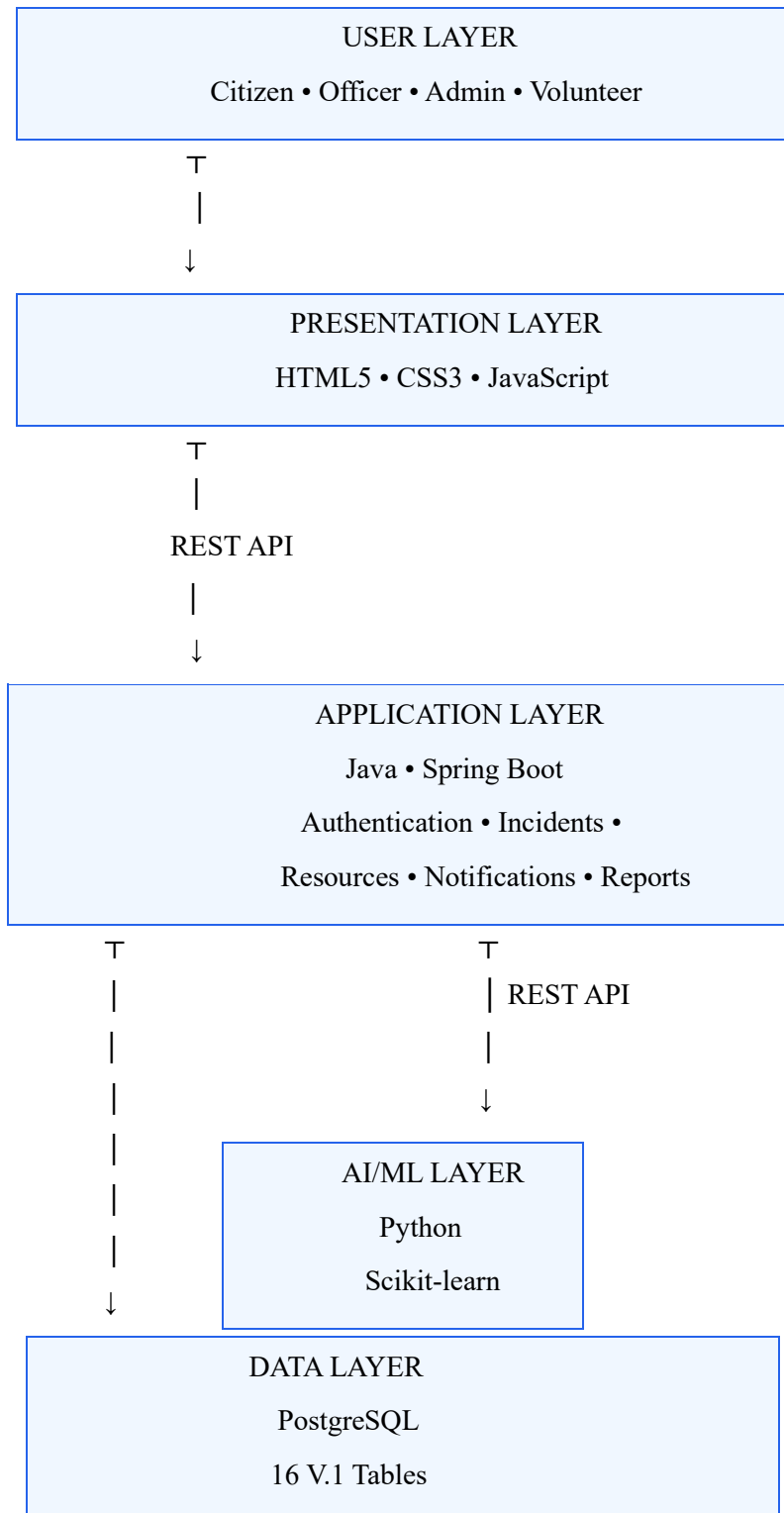
It provides:

- Remote source-code storage.
- Version history.
- Project collaboration.
- Backup.
- Documentation management.

The project repository can contain source code, database scripts, diagrams, documentation, and other project-related files.

2.8 Technology Architecture

The selected technologies can be organized into the following logical architecture:



This architecture separates the user interface, application logic, AI functionality, and data storage, making the system easier to maintain and extend.

2.9 Technology Selection Summary

Technology	Purpose
HTML5	Structure of the web interface
CSS3	Styling, layout and responsive design
JavaScript	Client-side interaction and dynamic functionality
Java	Backend programming and business logic
Spring Boot	Backend framework and REST API development
PostgreSQL	Relational database management
Python	AI/ML service development
Scikit-learn	Machine-learning implementation
REST API	Communication between system components
Visual Paradigm	UML, ERD, DFD and system modelling
Git	Version control
GitHub	Remote repository and project management

2.10 Justification of Technology Selection

The selected technology stack provides a practical combination of technologies for implementing the AI-Rescue platform.

HTML5, CSS3, and JavaScript provide the foundation for the web interface. **Java and Spring Boot** provide a structured and modular backend capable of implementing authentication, business logic, REST APIs, incident management, resource coordination, and notifications.

PostgreSQL is selected as the relational database because the project contains multiple interconnected entities requiring primary keys, foreign keys, constraints, transactions, and referential integrity.

Python and scikit-learn provide a suitable environment for developing the AI-assisted decision-support component. Keeping the AI component as a separate service allows it to communicate with the Java backend without tightly coupling the two technologies.

REST APIs provide standardized communication between the frontend, backend, and AI service. **Visual Paradigm** supports the creation of the project's system-design diagrams, while Git and GitHub support version control and project management.

Overall, the selected technologies provide a balance between **functionality, maintainability, scalability, security, modularity, and academic feasibility**.

CHAPTER 3: REQUIREMENTS AND ANALYSIS

This chapter defines the problem addressed by the **AI-Rescue: Intelligent Disaster Response & Resource Coordination Platform**, identifies the functional and non-functional requirements, describes the project planning and scheduling, specifies the required hardware and software environment, and presents the preliminary description and conceptual models of the proposed system.

3.1 Problem Definition

Natural disasters such as floods, earthquakes, cyclones, landslides, fires, and severe storms can affect large populations within a short period of time. During such situations, emergency authorities need accurate information about incidents, affected people, available resources, rescue teams, hospitals, and shelters.

A major challenge in disaster response is the **coordination of information and resources**. Incident reports may arrive from different sources, while emergency personnel may need to manually verify reports, determine their priority, identify available resources, and coordinate the response. This can result in delays, inefficient resource utilization, duplicate efforts, and difficulty in maintaining a centralized view of ongoing incidents.

The proposed **AI-Rescue** platform addresses this problem by providing a centralized web-based system for disaster incident reporting, incident verification, resource management, emergency coordination, notifications, and reporting.

The system also incorporates an **AI-assisted decision-support component**. After an incident has been verified by an authorized officer, the AI component analyzes relevant incident information and provides recommendations regarding severity, priority, and required resources. These recommendations are reviewed by the authorized officer before operational action is taken.

Therefore, the primary problem addressed by the project is:

To develop a centralized and intelligent disaster-response platform that improves incident management, resource coordination, communication, and decision support while reducing delays in emergency response operations.

3.2 Requirements Specification

The requirements of the AI-Rescue platform are divided into **functional requirements** and **non-functional requirements**.

3.2.1 Functional Requirements

FR-01: User Registration

The system shall allow new users to register by providing the required personal and account information.

FR-02: User Authentication

The system shall authenticate users using valid login credentials.

FR-03: Role-Based Access

The system shall provide different access permissions according to the user's assigned role.

Possible roles include:

- Citizen
- Volunteer / Support User
- Officer
- Administrator

FR-04: Disaster Incident Reporting

The system shall allow authorized users, particularly citizens, to submit disaster incident reports.

An incident report may contain:

- Disaster type.
- Incident title.
- Description.
- Location.
- Latitude and longitude.
- Number of people affected.
- Severity-related information.
- Supporting evidence.

FR-05: Incident Validation

The system shall validate the information entered during incident submission before storing the incident.

FR-06: Incident Verification

The system shall allow an authorized officer to review submitted incidents and determine whether an incident is genuine/valid.

Invalid incidents can be rejected and the relevant user can be notified.

FR-07: Incident Management

The system shall maintain the status and lifecycle of incidents from reporting through resolution.

FR-08: AI-Assisted Analysis

After an incident has been verified, the system shall send relevant incident information to the AI component for analysis.

The AI component shall provide decision-support information such as:

- Severity assessment.
- Priority recommendation.
- Required resource recommendation.
- Confidence score.

The AI output shall be treated as a **recommendation**, not as an automatic operational decision.

FR-09: AI Recommendation Review

The authorized officer shall be able to review the AI-generated recommendations before taking operational action.

FR-10: Resource Management

The system shall maintain information about available emergency resources.

Resources may include:

- Rescue resources.
- Vehicles.
- Equipment.
- Other response assets.

FR-11: Resource Availability

The system shall allow authorized users to check the availability and status of response resources.

FR-12: Resource Assignment

The system shall allow authorized officers or administrators to assign available resources to verified incidents.

The assignment may include vehicles, equipment, and other resources.

FR-13: Emergency Service Information

The system shall maintain information related to emergency facilities such as hospitals and shelters.

Hospital information may include emergency capacity and available beds, while shelter information may include capacity and current occupancy.

FR-14: Notifications

The system shall generate notifications for relevant users regarding important incident and operational events.

Examples include:

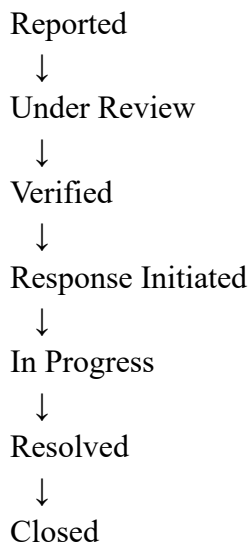
- New incident notification.

- Incident verification result.
- Resource assignment notification.
- Response-status update.
- Other emergency notifications.

FR-15: Incident Status Management

The system shall allow authorized users to update the status of an incident during the response process.

Possible states may include:



FR-16: Dashboard

The system shall provide dashboards for authorized users to monitor relevant disaster-response information.

The dashboard may display:

- Active incidents.
- Incident priorities.
- Resource availability.
- Response status.
- Notifications.
- Summary information.

FR-17: Report Generation

The system shall allow authorized users to generate reports based on stored system information.

FR-18: Audit Logging

The system shall record important system activities for traceability and accountability.

FR-19: System Settings

The administrator shall be able to manage relevant system configuration settings.

3.2.2 Non-Functional Requirements

NFR-01: Security

The system should protect user accounts and sensitive operational information through authentication, authorization, secure password storage, and role-based access control.

NFR-02: Performance

The system should respond to normal user requests within a reasonable time and should efficiently handle incident and resource data.

NFR-03: Reliability

The system should maintain data consistency and minimize failures during normal operation.

NFR-04: Usability

The interface should be simple and understandable so that users can perform required operations without unnecessary complexity.

NFR-05: Scalability

The architecture should allow additional users, incidents, resources, and future services to be incorporated without major changes to the overall design.

NFR-06: Maintainability

The application should use a modular architecture so that individual components can be modified or enhanced independently.

NFR-07: Data Integrity

The database should maintain accurate and consistent information using primary keys, foreign keys, constraints, and appropriate validation.

NFR-08: Availability

The system should be designed to remain accessible during normal operational periods and should support recovery from common application or database failures.

NFR-09: Extensibility

The architecture should allow future integration with technologies such as IoT devices, weather services, mapping services, mobile applications, and external emergency systems.

3.3 Planning and Scheduling

The development of AI-Rescue is divided into several stages to ensure that the system is developed systematically.

Table 3.1: Work Breakdown Structure & Execution Timeline (June 2026 – February 2027)

ID	Work Package / Activity	Start Date	End Date	Days	Category	Key Deliverable
01	Project Inception, Scope & Feasibility	23 Jun 2026	28 Jun 2026	6d	Planning & Initiation	Project Charter
02	Requirement Gathering & Tech Survey	29 Jun 2026	09 Jul 2026	11d	Planning & Analysis	Survey Report
03	Software Requirements Specification (SRS)	10 Jul 2026	19 Jul 2026	10d	Requirements Analysis	SRS Document
04	System Architecture & High-Level Design	20 Jul 2026	29 Jul 2026	10d	System Architecture	Architecture Blueprint
05	Database Design & ER Modeling (Master ERD)	30 Jul 2026	08 Aug 2026	10d	Database Engineering	ER Diagrams & DDL
06	Data Flow Diagrams (DFD Level 0 & Level 1)	09 Aug 2026	16 Aug 2026	8d	Process Modeling	DFD Specifications
07	UML Modeling (Use Case & Sequence Diagrams)	17 Aug 2026	25 Aug 2026	9d	Object-Oriented Design	UML Diagrams
08	System Flowchart & Module Logic Diagrams	26 Aug 2026	03 Sep 2026	9d	Procedural Design	Logic Flowcharts
09	UI/UX Wireframes & AI Architecture Spec	04 Sep 2026	12 Sep 2026	9d	Interface & AI Design	UI Prototypes
10	Phase 1 Documentation (Chapters 1 to 4)	23 Jun 2026	18 Sep 2026	88d	Academic Documentation	Draft Chapters 1–4
11	Review, Diagram Verification & Corrections	19 Sep 2026	24 Sep 2026	6d	Quality Assurance	Review Sign-off
12	Phase 1 Final Interim Submission & Freeze	25 Sep 2026	25 Sep 2026	1d	Milestone 1	Interim Report Submission
13	Dev Setup, Scaffolding & DB Migration	26 Sep 2026	08 Oct 2026	13d	Core Full-Stack Dev	Dev Environment & Baseline
14	Backend Services & REST APIs (Spring Boot)	09 Oct 2026	02 Nov 2026	25d	Backend Engineering	REST API Endpoints
15	Frontend Web & Mobile Portal (React.js UI)	24 Oct 2026	20 Nov 2026	28d	Frontend Engineering	Responsive UI Views
16	AI Decision-Support & Severity Index Engine	15 Nov 2026	10 Dec 2026	26d	AI & NLP Services	Triage & Ranking Model
17	GIS Interactive Mapping & Resource Dispatch	01 Dec 2026	24 Dec 2026	24d	Geo-Spatial & Dispatch	GIS Map & Routing
18	Shelter, Hospital & Notification Services	18 Dec 2026	08 Jan 2027	22d	Integration Services	Alert & Capacity Engines
19	System Integration & End-to-End Testing	05 Jan 2027	25 Jan 2027	21d	Quality Assurance	Test Case Execution Report
20	User Acceptance Testing & Cloud Deployment	22 Jan 2027	08 Feb 2027	18d	Deployment & Hosting	Production Cloud URL

21	Phase 2 Documentation (Chapters 5, 6 & 7)	15 Jan 2027	18 Feb 2027	35d	Academic Documentation	Final Chapters 5–7
22	Final Project Submission & Degree Defense	26 Feb 2027	28 Feb 2027	3d	Milestone 2	Project Defense / Viva

The project schedule was prepared using a Gantt chart to organize the major activities of the AI-Rescue project from initial planning through final documentation and submission.

The project schedule was prepared using an academic Gantt chart to organize and track the major work packages of the AI-Rescue platform across both Phase 1 (Planning, Requirement Analysis, Architecture & System Design) and Phase 2 (Full-Stack Implementation, AI Priority Engine, GIS Services, Quality Assurance, Cloud Deployment, Final Documentation and Viva Defense).

Figure 3.1: Gantt Chart Phase 1 Diagram

Figure 3.2: Gantt Chart Phase 2 Diagram

The project schedule is structured into two standalone execution phases. Phase 1 (Figure 3.1) encompasses initial problem definition, requirements engineering, SRS specification, architectural modeling (DFDs, ERDs, UML diagrams), and Chapters 1–4 interim documentation, concluding on 25 September 2026. Phase 2 (Figure 3.2) initiates immediately on 26 September 2026 without interruption, covering core Spring Boot backend services, PostgreSQL database schema migrations, React web portal development, AI decision-support algorithms, GIS routing, automated testing, cloud deployment, and final thesis defense by 28 February 2027.

The Gantt chart shows the planned timeline and duration of each major project activity. Activities are arranged according to their dependencies, with overlapping tasks where appropriate.

3.4 Software and Hardware Requirements

3.4.1 Software Requirements

The proposed development environment requires the following software:

Software	Purpose
Operating System	Windows / Linux
Java JDK	Java application development
Spring Boot	Backend development
Python	AI/ML development
Scikit-learn	Machine learning
PostgreSQL	Database management
Visual Paradigm	System modelling and diagrams
Web Browser	Application testing
Git	Version control
GitHub	Repository management
IDE	Java/Python application development

The exact IDE can be selected according to development convenience.

3.4.2 Hardware Requirements

A standard modern computer system is sufficient for developing the academic version of the platform.

Minimum Recommended Configuration

Component	Requirement
Processor	Intel Core i5 / equivalent
RAM	8 GB minimum
Storage	256 GB or higher
Network	Internet connection
Display	Standard monitor
Input	Keyboard and mouse

Recommended Configuration

Component	Requirement
Processor	Intel Core i5/i7 or equivalent
RAM	16 GB
Storage	512 GB SSD
Network	Stable broadband connection

A dedicated high-end GPU is **not required for the initial academic version** if the AI component uses conventional machine-learning models rather than large deep-learning models.

3.5 Preliminary Product Description

The **AI-Rescue** platform is proposed as a centralized web-based disaster management and response coordination system.

The platform provides a common environment where different stakeholders can interact with disaster-related information according to their roles.

The major capabilities of the system include:

1. User Management

Users can register and authenticate themselves. Role-based permissions determine the operations available to each user.

2. Incident Management

Citizens can report disaster incidents by providing structured information and optional supporting evidence. Officers can review, verify, reject, and manage incidents.

3. AI Decision Support

Once an incident has been verified, the AI module analyzes relevant information and generates recommendations related to severity, priority, and required resources.

4. Resource Coordination

Authorized officers can view available resources and assign appropriate resources to verified incidents.

5. Hospital and Shelter Information

The platform maintains emergency-service information such as hospital capacity, available beds, shelter capacity, and current occupancy.

6. Notification Management

The system provides notifications to relevant users based on incident and operational events.

7. Dashboard and Reporting

Authorized users can monitor disaster-response activities through dashboards and generate reports from stored information.

8. Audit and Administration

Important system activities are recorded through audit logs, while administrators can manage system configuration.

3.6 Conceptual Models

Conceptual models provide a high-level representation of how the proposed system operates and how its components interact.

The major conceptual models prepared for AI-Rescue are:

1. **Data Flow Diagram (DFD)**
2. **Use Case Diagram**

These models describe different aspects of the same system.

3.6.1 Data Flow Concept

At a conceptual level, information flows through the system as follows:

1. DFD Level 0

The Level 0 Data Flow Diagram provides a high-level view of the AI-Rescue system and shows how data is exchanged between external entities and the system.

DFD LEVEL 0 – AI-POWERED SMART DISASTER RESPONSE & RESOURCE COORDINATION SYSTEM

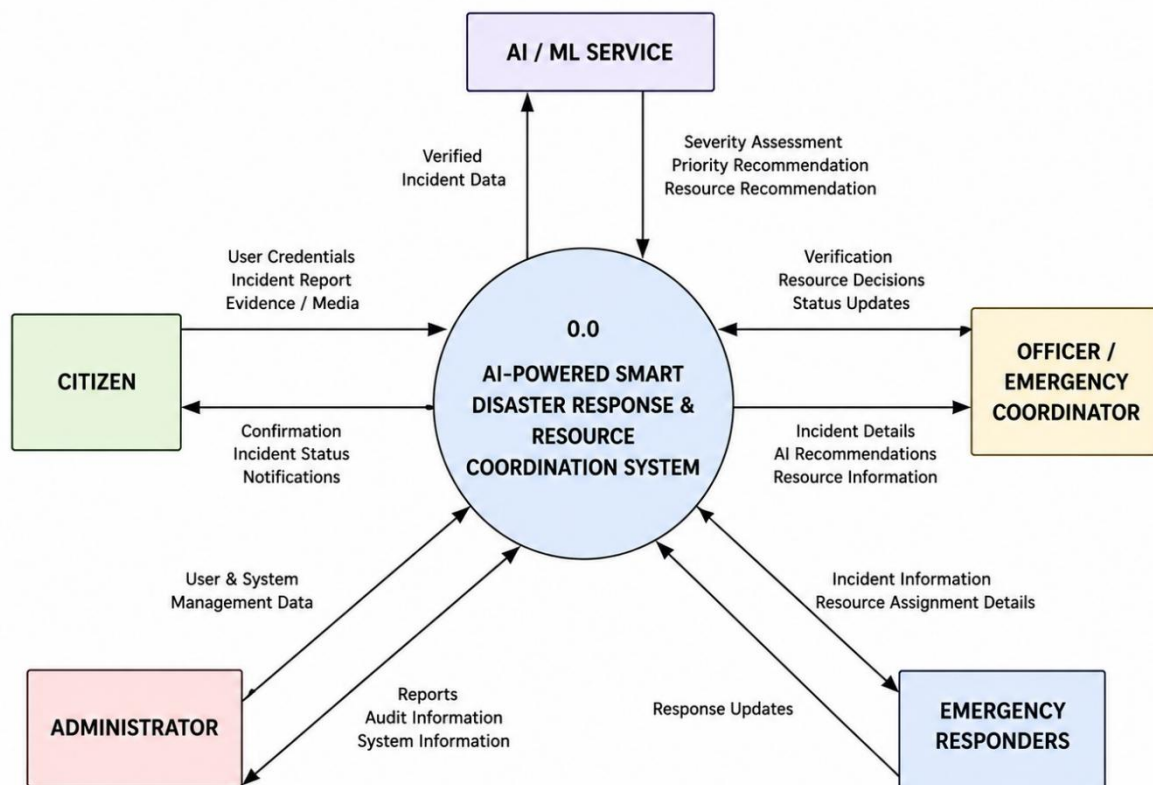


Figure 3.3: Data Flow Diagram – Level 0

The diagram represents the major input and output data flows between users and the disaster response platform.

2. DFD Level 1

The Level 1 DFD decomposes the major processes of the AI-Rescue system into more detailed processes.

DFD LEVEL 1 – AI-POWERED SMART DISASTER RESPONSE & RESOURCE COORDINATION SYSTEM

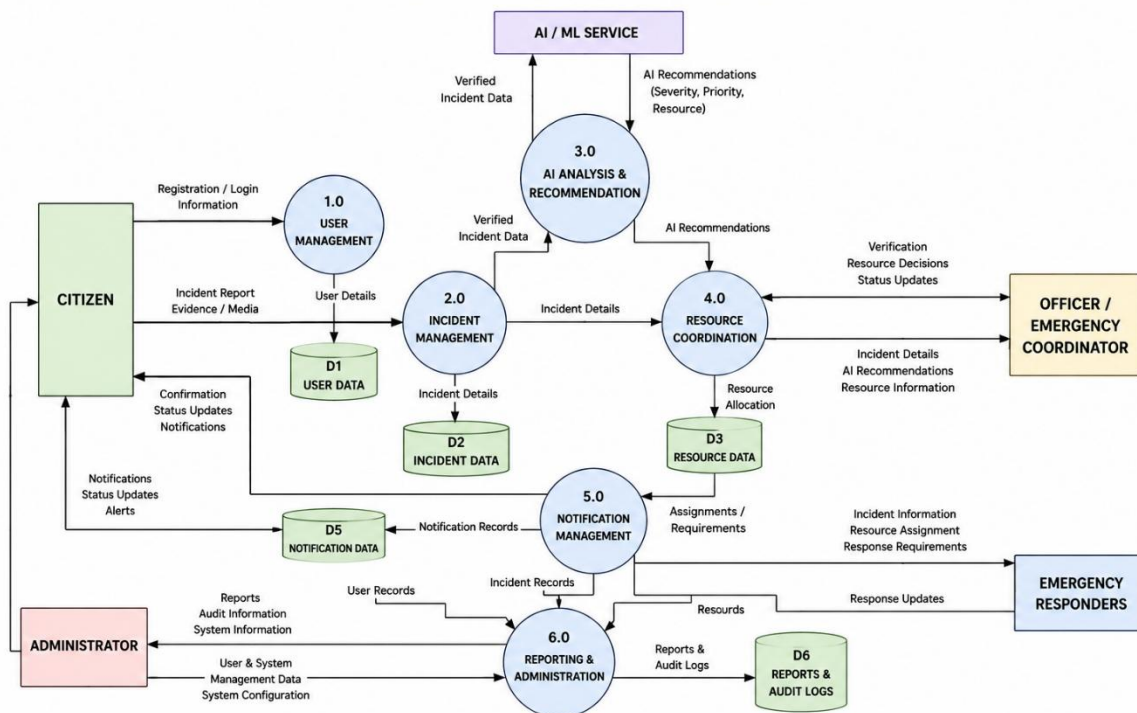


Figure 3.4: Data Flow Diagram – Level 1

This expands the major processes of the Level 0 DFD.

It illustrates the flow of incident information, verification data, AI recommendations, resource information, notifications, and response updates within the system.

3.6.2 Use Case Diagram

The Use Case Diagram represents the interactions between different users and the AI-Rescue system. It identifies the major actors and the functions available to each actor.

Figure 3.5: UML Use Case Diagram

The diagram shows the major system functions performed by citizens, volunteers, officers, hospital and shelter personnel, and administrators according to their assigned roles.

CHAPTER 4: SYSTEM DESIGN

4.1 Basic Modules

The AI-Rescue platform is divided into functional modules. Each module is responsible for a specific set of operations while communicating with other modules through the application's backend.

4.1.1 User Authentication and Role Management

This module manages user registration, login, authentication, and role-based access.

Main functions:

- User registration.
- User login.
- Credential verification.
- Role identification.
- Session management.
- Access control.
- Account status management.

The system provides different permissions according to the user's role.

Related tables:

- ROLE
 - USER
-

4.1.2 Disaster Incident Management

This is the central operational module of the system.

It allows users to report disaster incidents and allows authorized officers to review and manage them.

Main functions:

- Create incident reports.
- Enter disaster details.

- Record incident location.
- Record affected population.
- Attach supporting evidence.
- Validate incident information.
- Review submitted incidents.
- Verify or reject incidents.
- Update incident status.
- Close resolved incidents.

Related tables:

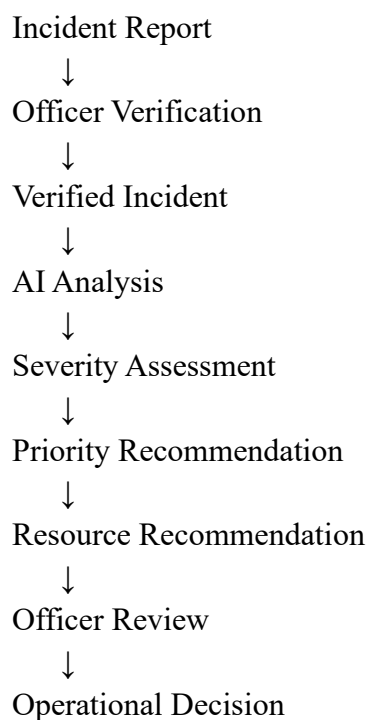
- DISASTER_TYPE
- INCIDENT
- INCIDENT_MEDIA
- USER

4.1.3 AI Decision-Support Module

The AI module assists authorized officers in evaluating verified disaster incidents.

The AI module is **not responsible for independently approving incidents or dispatching emergency resources.**

The workflow is:



The module may generate:

- Severity score.
- Priority recommendation.
- Recommended resource types.
- Confidence score.
- Reasoning/explanation associated with the recommendation.

Related table:

- AI_RECOMMENDATION

4.1.4 Resource Management Module

This module manages the emergency resources available to the organization.

Resources can include general response resources, vehicles, and equipment.

Main functions:

- Add resources.
- Update resource information.
- View resource availability.
- Track resource status.
- Manage vehicles.
- Manage equipment.
- Assign resources to incidents.

Related tables:

- RESOURCE
- VEHICLE
- EQUIPMENT
- RESOURCE_ASSIGNMENT

4.1.5 Resource Assignment and Coordination Module

This module connects verified incidents with the resources required for response.

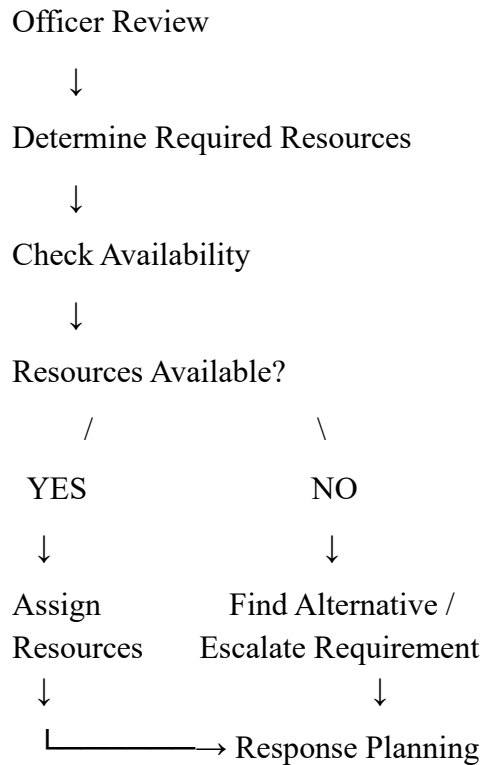
After the officer reviews the AI recommendation, the system checks available resources.

Verified Incident



AI Recommendation





If suitable resources are unavailable, the system can record the requirement for alternative resources or escalation to a higher-level authority/support mechanism.

Related tables:

- RESOURCE
- VEHICLE
- EQUIPMENT
- RESOURCE_ASSIGNMENT
- USER

4.1.6 Hospital Management Module

This module maintains information about hospitals that may be relevant during emergency response.

The information may include:

- Hospital name.
- Address.
- Location.
- Contact number.
- Emergency capacity.
- Available beds.

- Current status.

The module helps authorized users identify hospital capacity when emergency medical support is required.

Related table:

- HOSPITAL
-

4.1.7 Shelter Management Module

This module manages emergency shelter information.

The information may include:

- Shelter name.
- Address.
- Location.
- Contact number.
- Total capacity.
- Current occupancy.
- Availability/status.

The information can support decisions regarding temporary accommodation for affected populations.

Related table:

- SHELTER
-

4.1.8 Notification Module

The notification module manages communication of important system events to relevant users.

Notifications can be generated for events such as:

- New incident submission.
- Incident verification.
- Incident rejection.
- Resource assignment.
- Response updates.
- Other important operational events.

Related tables:

- NOTIFICATION

- USER
 - INCIDENT
-

4.1.9 Dashboard and Reporting Module

The dashboard provides an overview of disaster-response activities.

Possible information displayed includes:

- Total incidents.
- Active incidents.
- Incident priority.
- Incident status.
- Resource availability.
- Response activities.
- Notifications.
- Summary statistics.

The reporting component allows authorized users to generate reports based on stored system information.

Related tables:

- REPORT_LOG
 - INCIDENT
 - RESOURCE
 - RESOURCE_ASSIGNMENT
-

4.1.10 Audit and System Administration Module

This module provides administrative control and system traceability.

The audit component records important actions performed within the system.

Examples include:

- User-related changes.
- Incident modifications.
- Resource assignments.
- Administrative actions.
- Configuration changes.

The administrator can also manage system-level settings.

Related tables:

- AUDIT_LOG
- SYSTEM_SETTINGS
- USER

4.2 Data Design

Data design defines how information is organized, stored, related, and protected within the AI-Rescue platform.

The proposed system uses a **relational database model with PostgreSQL**.

The finalized V1 database consists of **16 tables**.

These tables are organized into the following logical groups:

Category	Tables
Identity	ROLE, USER
Incident Management	DISASTER_TYPE, INCIDENT, INCIDENT_MEDIA
AI	AI_RECOMMENDATION
Resource Management	RESOURCE, VEHICLE, EQUIPMENT, RESOURCE_ASSIGNMENT
Emergency Services	HOSPITAL, SHELTER
Communication	NOTIFICATION
Administration	REPORT_LOG, AUDIT_LOG, SYSTEM_SETTINGS

4.2.1 Schema Design

1. ER DIAGRAM

Figure 4.1: Entity Relationship Diagram

1. ROLE

Stores the roles available within the system.

Field	Description
role_id	Primary Key
role_name	Name of the role
description	Description of the role
created_at	Creation timestamp

2. USER

Stores registered users and their role information.

Field	Description
user_id	Primary Key
role_id	Foreign Key → ROLE
full_name	User's name
email	User email
phone	Contact number
password_hash	Encrypted/hashed password
address	User address
city	City
state	State
status	Account status
created_at	Creation timestamp
updated_at	Last update timestamp
last_login	Last login timestamp

3. DISASTER_TYPE

Stores the different types of disasters supported by the system.

Field	Description
disaster_type_id	Primary Key
type_name	Disaster type
description	Description

Field	Description
is_active	Active/inactive status
created_at	Creation timestamp

4. INCIDENT

Stores the main disaster incident information.

Field	Description
incident_id	Primary Key
disaster_type_id	Foreign Key → DISASTER_TYPE
reported_by	Foreign Key → USER
verified_by	Foreign Key → USER
title	Incident title
description	Incident description
location_address	Incident location
latitude	Geographic latitude
longitude	Geographic longitude
reported_at	Report timestamp
verified_at	Verification timestamp
severity_level	Severity classification
priority_level	Priority classification
people_affected	Number of affected people
status	Incident status
resolution_notes	Resolution information
resolved_at	Resolution timestamp
created_at	Creation timestamp
updated_at	Update timestamp

5. INCIDENT_MEDIA

Stores evidence associated with an incident.

Field	Description
media_id	Primary Key
incident_id	Foreign Key → INCIDENT
uploaded_by	Foreign Key → USER

Field	Description
media_type	Type of media
file_path	Stored file location
caption	Media description
uploaded_at	Upload timestamp
is_verified	Verification status

6. AI_RECOMMENDATION

Stores the output generated by the AI decision-support component.

Field	Description
recommendation_id	Primary Key
incident_id	Foreign Key → INCIDENT
model_name	AI model used
severity_score	AI-generated severity score
priority_level	Recommended priority
recommended_resources	Recommended resources
reasoning	Explanation of recommendation
confidence_score	AI confidence
created_at	Creation timestamp

7. RESOURCE

Stores general emergency resources.

Field	Description
resource_id	Primary Key
resource_name	Resource name
resource_type	Resource category
description	Description
quantity_available	Available quantity
location	Current location
status	Resource status
contact_number	Contact information
created_at	Creation timestamp
updated_at	Update timestamp

8. VEHICLE

Stores emergency-response vehicles.

Field	Description
vehicle_id	Primary Key
vehicle_number	Vehicle registration/identifier
vehicle_type	Vehicle category
capacity	Carrying/response capacity
current_location	Current location
status	Vehicle status
assigned_team	Assigned team
created_at	Creation timestamp
updated_at	Update timestamp

9. EQUIPMENT

Stores emergency equipment.

Field	Description
equipment_id	Primary Key
equipment_name	Equipment name
equipment_type	Equipment category
quantity_total	Total quantity
quantity_available	Available quantity
condition_status	Equipment condition
location	Current location
created_at	Creation timestamp
updated_at	Update timestamp

10. RESOURCE_ASSIGNMENT

Stores the assignment of resources to disaster-response activities.

Field	Description
assignment_id	Primary Key
incident_id	Foreign Key → INCIDENT
resource_id	Foreign Key → RESOURCE

Field	Description
vehicle_id	Foreign Key → VEHICLE
equipment_id	Foreign Key → EQUIPMENT
assigned_by	Foreign Key → USER
assigned_at	Assignment timestamp
start_time	Start of assignment
end_time	End of assignment
status	Assignment status
notes	Additional information

11. HOSPITAL

Stores emergency hospital information.

Field	Description
hospital_id	Primary Key
hospital_name	Hospital name
address	Hospital address
latitude	Geographic latitude
longitude	Geographic longitude
contact_number	Contact number
emergency_capacity	Emergency capacity
available_beds	Available beds
status	Hospital status
created_at	Creation timestamp
updated_at	Update timestamp

12. SHELTER

Stores emergency shelter information.

Field	Description
shelter_id	Primary Key
shelter_name	Shelter name
address	Shelter address
latitude	Geographic latitude
longitude	Geographic longitude

Field	Description
contact_number	Contact number
capacity	Total capacity
current_occupancy	Current occupants
status	Shelter status
created_at	Creation timestamp
updated_at	Update timestamp

13. NOTIFICATION

Stores notifications generated by the system.

Field	Description
notification_id	Primary Key
user_id	Foreign Key → USER
incident_id	Foreign Key → INCIDENT
notification_type	Notification category
title	Notification title
message	Notification message
sent_at	Sent timestamp
read_at	Read timestamp
status	Notification status

14. REPORT_LOG

Stores information about generated reports.

Field	Description
report_id	Primary Key
generated_by	Foreign Key → USER
report_type	Report category
report_title	Report title
parameters	Report parameters
file_path	Generated file location
generated_at	Generation timestamp

15. AUDIT_LOG

Stores records of important system activities.

Field	Description
audit_id	Primary Key
user_id	Foreign Key → USER
action	Action performed
table_name	Affected table
record_id	Affected record
old_values	Previous values
new_values	Updated values
ip_address	Source IP
created_at	Timestamp

16. SYSTEM_SETTINGS

Stores configurable system settings.

Field	Description
setting_id	Primary Key
updated_by	Foreign Key → USER
setting_key	Setting identifier
setting_value	Setting value
description	Setting description
updated_at	Update timestamp

2. Database Design

Figure 4.2: Database Design Diagram

The database design represents the relational structure of the AI-Rescue platform. The system uses 16 tables to store and manage operational data.

The database contains tables for user management, incidents, AI recommendations, resources, emergency facilities, notifications, reporting, auditing, and system configuration. Primary and foreign keys maintain relationships and data integrity.

4.2.2 Data Integrity and Constraints

Data integrity ensures that the information stored in the database remains accurate, consistent, and reliable.

The following constraints are considered in the database design.

Primary Key

Every table contains a unique primary key that identifies each record.

For example:

USER.user_id

INCIDENT.incident_id

RESOURCE.resource_id

HOSPITAL.hospital_id

Foreign Key

Foreign keys maintain relationships between related tables.

For example:

USER.role_id

↓

ROLE.role_id

and:

INCIDENT.disaster_type_id

↓

DISASTER_TYPE.disaster_type_id

NOT NULL

Important attributes should not be allowed to contain empty values.

For example, a user account should require essential information such as name, email, and password.

UNIQUE

Attributes such as email addresses can be restricted to unique values where required.

CHECK Constraints

Appropriate values can be restricted using validation rules.

For example:

people_affected >= 0

quantity_available >= 0

available_beds >= 0

current_occupancy >= 0

Referential Integrity

Foreign-key relationships prevent invalid references between related records.

Transaction Integrity

Database transactions can be used when multiple related operations must either complete successfully together or be rolled back.

4.3 Procedural Design

Procedural design describes the logical procedures followed by the system when processing different operations.

The major procedures include:

1. User authentication.
 2. Incident reporting.
 3. Incident verification.
 4. AI analysis.
 5. Resource availability checking.
 6. Resource assignment.
 7. Incident response and status management.
 8. Notification processing.
 9. Report generation.
-

4.3.1 Logic Diagrams

Figure 4.3: Overall System Flowchart Diagram

The overall system flowchart represents the complete operational workflow of the AI-Rescue platform, from incident reporting to incident resolution.

The workflow includes authentication, incident reporting, validation, officer verification, AI-assisted analysis, resource allocation, emergency response, status updates, and incident closure.

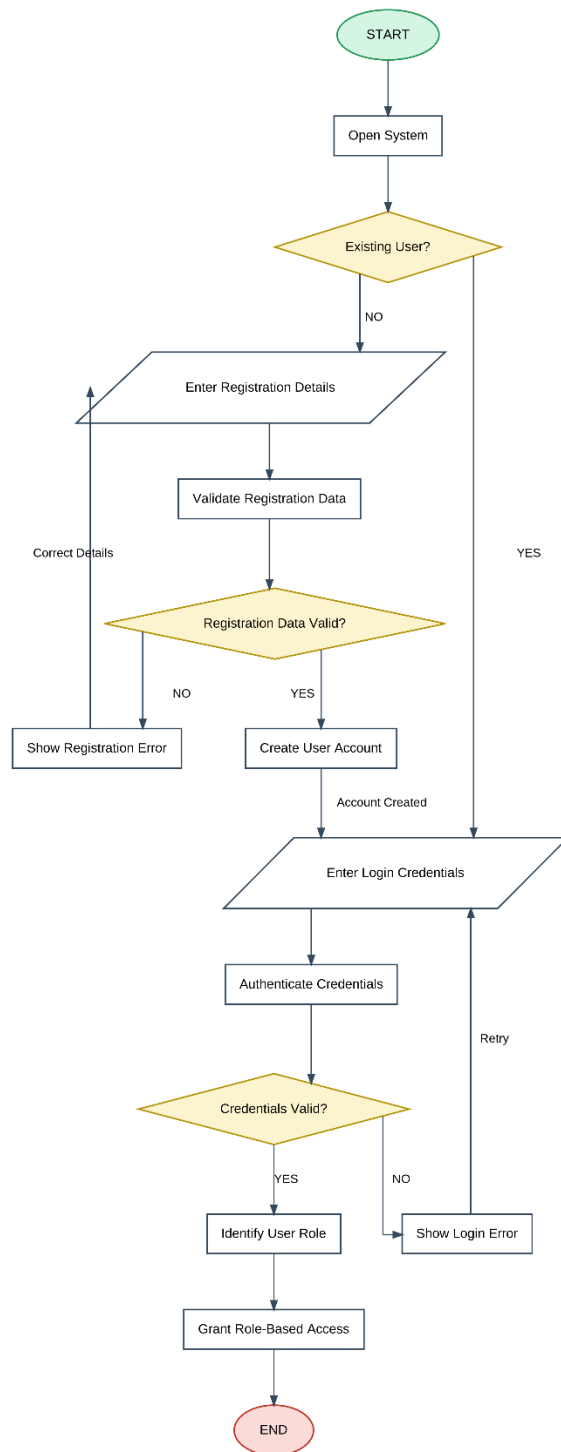


Figure 4.4: User Authentication Module Flowchart

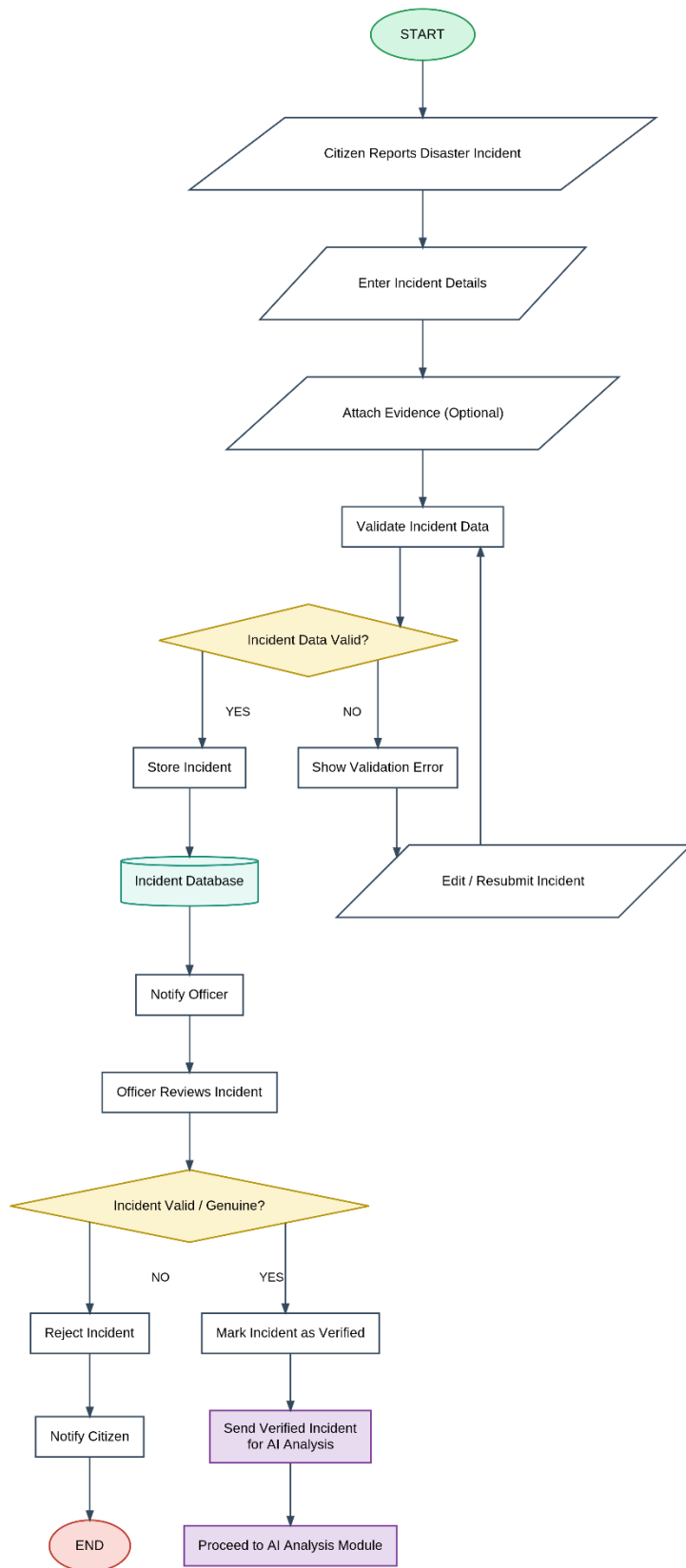


Figure 4.5: Incident Reporting Flowchart

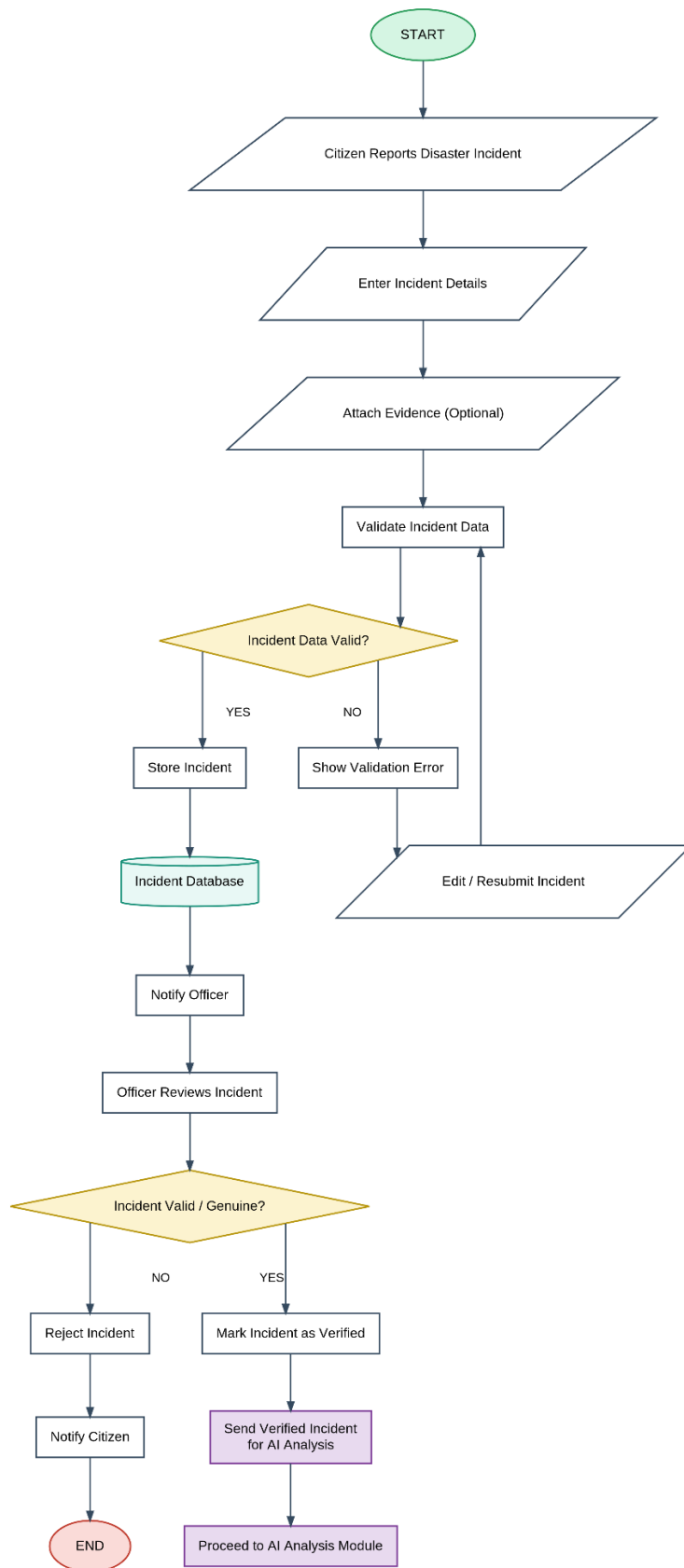


Figure 4.6: AI Analysis Flowchart

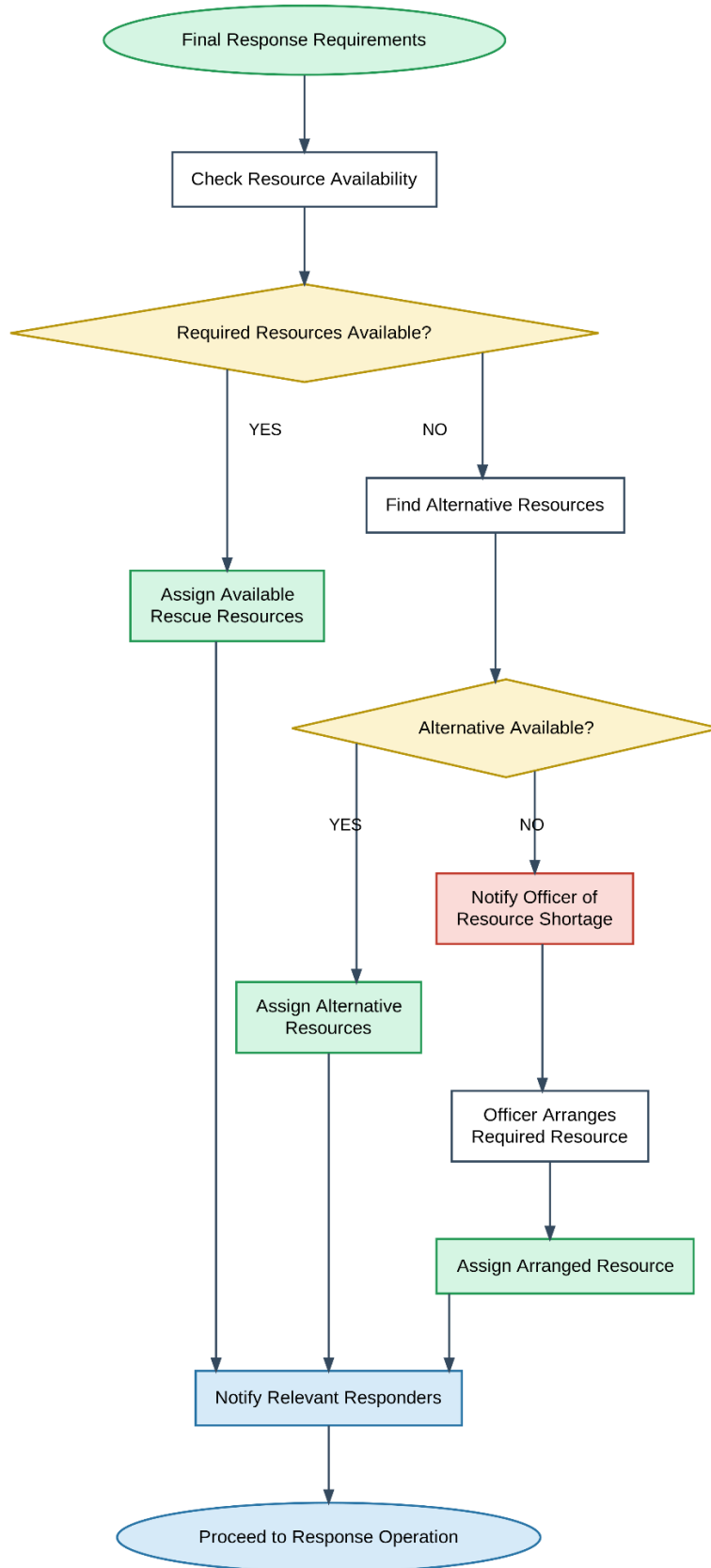


Figure 4.7: Resource Management and Assignment Module Flowchart

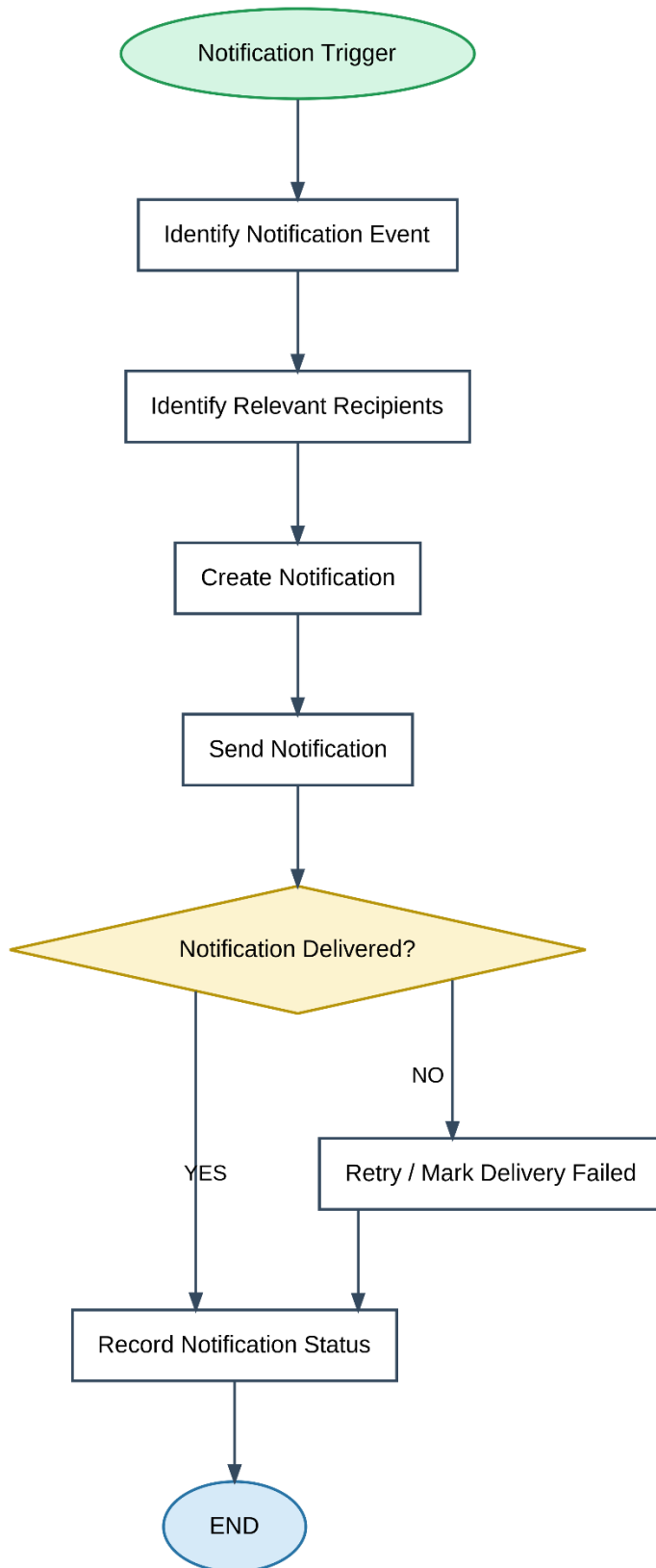


Figure 4.8: Notification Module Flowchart

The Sequence Diagram models the chronological procedural interaction between platform actors (Citizen, Response Officer, System Administrator) and core backend services (Authentication, Incident Service, AI Severity Engine, Resource Dispatcher, and Notification Engine) across the entire incident lifecycle.

Figure 4.9: Sequence Diagram

4.3.2 Data Structures

The system uses several logical data structures during processing.

User Object

User

- └─ user_id
- └─ role_id
- └─ full_name
- └─ email
- └─ phone
- └─ status

Incident Object

Incident

- └─ incident_id
- └─ disaster_type_id
- └─ reported_by
- └─ location
- └─ severity
- └─ priority
- └─ people_affected
- └─ status

AI Recommendation Object

AI Recommendation

- └─ recommendation_id
- └─ incident_id
- └─ severity_score
- └─ priority_level
- └─ recommended_resources

└── confidence_score

Resource Assignment Object

Resource Assignment

└── assignment_id

└── incident_id

└── resource_id

└── vehicle_id

└── equipment_id

└── assigned_by

└── status

These structures represent the logical information handled by the application before or during database operations.

4.3.3 Algorithms Design

Algorithm 1: User Authentication

Step 1: Accept email and password.

Step 2: Validate the entered credentials.

Step 3: Search for the corresponding user account.

Step 4: Verify the password against the stored password hash.

Step 5: Check account status.

Step 6: If valid, create an authenticated session/token.

Step 7: Identify the user's role.

Step 8: Provide access according to the assigned role.

Step 9: Otherwise, display an authentication error.

Algorithm 2: Incident Verification

Step 1: Receive submitted incident.

Step 2: Display incident information to authorized officer.

Step 3: Officer reviews incident details and evidence.

Step 4: Determine whether the incident is genuine.

Step 5: If invalid:

 Reject incident.

Update incident status.

Notify citizen.

Step 6: If valid:

Mark incident as verified.

Record verification details.

Send incident for AI analysis.

Algorithm 3: AI-Assisted Incident Analysis

Step 1: Receive verified incident data.

Step 2: Prepare relevant input attributes.

Step 3: Send the data to the AI service.

Step 4: AI processes the incident information.

Step 5: Calculate/estimate severity.

Step 6: Generate priority recommendation.

Step 7: Generate required-resource recommendation.

Step 8: Calculate confidence score.

Step 9: Return recommendation to the backend.

Step 10: Store the recommendation.

Step 11: Present the recommendation to the authorized officer.

Algorithm 4: Resource Availability and Assignment

Step 1: Receive verified incident and required resource information.

Step 2: Check available resources.

Step 3: Check vehicle availability.

Step 4: Check equipment availability.

Step 5: If suitable resources are available:

Select appropriate resources.

Create resource assignment.

Update resource status.

Notify relevant responders.

Step 6: If required resources are unavailable:

Identify possible alternatives.

Record the unmet requirement.

Escalate the requirement when necessary.

Step 7: Continue response planning.

4.4 User Interface Design

The user interface is designed to provide a simple and role-specific experience.

4.4.1 Login Page

The login interface provides:

- Email/username input.
 - Password input.
 - Login button.
 - Registration option.
 - Error messages.
-

4.4.2 Citizen Dashboard

The citizen dashboard can provide:

- Report disaster button.
 - Submitted incidents.
 - Incident status.
 - Notifications.
 - Profile information.
-

4.4.3 Officer Dashboard

The officer dashboard provides access to operational information such as:

- New incidents.
- Pending verification.
- Verified incidents.
- AI recommendations.
- Resource availability.
- Active responses.
- Notifications.
- Reports.

4.4.4 Administrator Dashboard

The administrator interface provides:

- User management.
- Role management.
- Resource management.
- System settings.
- Audit records.
- Reports.

4.4.5 Incident Reporting Interface

The incident reporting form can contain:

- Disaster type.
- Incident title.
- Description.
- Location.
- Geographic coordinates.
- People affected.
- Supporting evidence.
- Submit button.

The interface should clearly indicate mandatory and optional fields.

4.5 Security Issues

Because the platform handles user information and potentially sensitive disaster-response information, security is an important part of the system design.

4.5.1 Authentication

Users must authenticate before accessing protected functionality.

4.5.2 Password Security

Passwords should not be stored as plain text. Passwords should be stored using secure password hashing mechanisms.

4.5.3 Role-Based Access Control

Users should only access functions permitted by their roles.

For example:

Citizen



Incident Reporting

Officer



Incident Verification

AI Recommendation Review

Resource Assignment

Administrator



System Management

4.5.4 Authorization

Authentication confirms who the user is, while authorization determines what the user is allowed to do.

4.5.5 Input Validation

User input should be validated to reduce invalid data and common security risks.

4.5.6 Secure Communication

Communication between clients and servers should use HTTPS in the deployed system.

4.5.7 Audit Logging

Important actions should be recorded so that system activities can be traced.

4.5.8 Database Security

Database credentials should be protected, and application users should receive only the permissions required for their operations.

4.6 Test Cases Design

Testing is required to verify that the system behaves according to its functional requirements.

The following test cases represent the major V1 functionality.

TC ID	Test Case	Input/Condition	Expected Result
TC-01	User Registration	Valid registration details	Account created successfully
TC-02	User Registration	Duplicate email	Registration rejected
TC-03	Login	Valid credentials	User authenticated

TC ID	Test Case	Input/Condition	Expected Result
TC-04	Login	Invalid password	Error displayed
TC-05	Role Access	Citizen accesses officer function	Access denied
TC-06	Incident Reporting	Valid incident details	Incident submitted
TC-07	Incident Validation	Missing mandatory field	Validation error
TC-08	Incident Verification	Officer marks incident valid	Incident becomes verified
TC-09	Incident Verification	Officer rejects incident	Incident rejected and notification generated
TC-10	AI Analysis	Verified incident	AI recommendation generated
TC-11	AI Analysis	Unverified incident	AI analysis not initiated
TC-12	Resource Availability	Available resources	Resources can be assigned
TC-13	Resource Availability	No suitable resource	Alternative/escalation process initiated
TC-14	Resource Assignment	Valid assignment	Assignment stored and resource status updated
TC-15	Notification	Operational event occurs	Relevant notification generated
TC-16	Incident Resolution	Incident resolved	Incident closed
TC-17	Report Generation	Valid report request	Report generated
TC-18	Audit Logging	Important system action	Audit record created
TC-19	Hospital Information	Update hospital capacity	Updated information stored
TC-20	Shelter Information	Update shelter occupancy	Updated information stored

CHAPTER 4 SUMMARY

The system design defines the architecture, modules, database structure, processing logic, algorithms, interface design, security considerations, and testing approach of the AI-Rescue platform.